```
        logprobs.append(logprob_)
        state_, _, done, info =
worker_env.step(action.detach().numpy())
        state = torch.from_numpy(state_).float()
        if done:
6
            reward = -10
            worker_env.reset()
        else:
            reward = 1.0
        rewards.append(reward)
return values, logprobs, rewards
```

- ***1*** Converts the environment state from a numpy array to a PyTorch tensor
- ***2*** Creates lists to store the computed state values (critic), log probabilities (actor), and rewards
- ***3*** Keeps playing the game until the episode ends
- ***4*** Computes the state value and log probabilities over actions
- ***5*** Using the actor's log probabilities over actions, creates and samples from a categorical distribution to get an action
- ***6*** If the last action caused the episode to end, sets the reward to –10 and resets the environment

The `run_episode` function just runs through a single episode of CartPole and collects the computed state values from the critic, log probabilities over actions from the actor, and rewards from the environment. We store these in lists and use them to compute our loss function later. Since this is an actor-critic method and not Q-learning, we take actions by directly sampling from the policy rather than arbitrarily choosing a policy like epsilon-greedy in Q-learning. There's nothing too out of the ordinary in this function, so let's move on to the updating function.

Listing 5.8. Computing and minimizing the loss

```
def
update_params(worker_opt,values,logprobs,rewards,clc=0.1,gamma=0.95
):
        rewards = torch.Tensor(rewards).flip(dims=(0,)).view(-1)
1
        logprobs = torch.stack(logprobs).flip(dims=(0,)).view(-1)
        values = torch.stack(values).flip(dims=(0,)).view(-1)
        Returns = []
```